

A Practical Guide to Layer normalization

TOPIC -- LAYER NORMALIZATION

-- 01 --

$$\mu_i(l) = \frac{1}{B} \sum_{b=1}^B x_{(b),i}^{(l)} \quad \sigma_i(l) = \sqrt{\frac{1}{B} \sum_{b=1}^B (x_{(b),i}^{(l)} - \mu_i(l))^2}$$

-- 02 --

γ and β allow the network to learn to undo the normalization, if this is beneficial. BatchNorm can be interpreted as removing the purely linear transformations, so that its layers focus solely on modelling the nonlinear aspects of data, which may be beneficial, as a neural network can always be augmented with a linear transformation layer on top. It is claimed in the original publication that BatchNorm works by reducing internal covariance shift, though the claim has both supporters and detractors.

-- 03 --

$$y_{h,w,c}^{(l)} = \sigma_c(l) \cdot x_{h,w,c}^{(l)} + \mu_c(l) \quad \text{style} \quad \left(\frac{x_{h,w,c}^{(l)}}{\sigma_c(l)} + \beta_c(l) \right) \cdot \gamma_c(l)$$

-- 04 --

$b_c(l)$ is the bias term for the c -th channel of the l -th layer. In order to preserve the translational invariance, BatchNorm treats all outputs from the same kernel in the same batch as more data in a batch. That is, it is applied once per kernel c (equivalently, once per channel c), not per activation $x_{h,w,c}^{(l+1)}$:

-- 05 --

Group normalization (GroupNorm) is a technique also solely used for CNNs. It can be understood as the LayerNorm for CNN applied

once per channel group. Suppose at a layer l , there are channels $1, 2, \dots, C$, then it is partitioned into groups $g_1, g_2, \dots, g_G$. Then, LayerNorm is applied to each group.

-- 06 --

$$\mu = \frac{1}{D} \sum_{i=1}^D x_i, \sigma^2 = \frac{1}{D} \sum_{i=1}^D (x_i - \mu)^2$$

-- 07 --

$$y_{(b),i}^{(l)} = \gamma_{(b),i}^{(l)} x_{(b),i}^{(l)} + \beta_{(b),i}^{(l)}$$

-- 08 --

$$\begin{aligned} \mu_{c,l} &= \frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W x_{h,w,c,l} (\sigma_{c,l})^2 = \frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W (x_{h,w,c,l} - \mu_{c,l})^2 \\ \sigma_{c,l}^2 &= \frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W (x_{h,w,c,l} - \mu_{c,l})^2 \\ \gamma_{c,l} &= \frac{\hat{x}_{c,l}}{\sqrt{\sigma_{c,l}^2 + \epsilon}} \\ \beta_{c,l} &= \gamma_{c,l} \end{aligned}$$

-- 09 --

$$\begin{aligned} \mu_{c,l} &= \frac{1}{BHW} \sum_{b=1}^B \sum_{h=1}^H \sum_{w=1}^W x_{b,h,w,c,l} (\sigma_{c,l})^2 = \frac{1}{BHW} \sum_{b=1}^B \sum_{h=1}^H \sum_{w=1}^W (x_{b,h,w,c,l} - \mu_{c,l})^2 \\ \sigma_{c,l}^2 &= \frac{1}{BHW} \sum_{b=1}^B \sum_{h=1}^H \sum_{w=1}^W (x_{b,h,w,c,l} - \mu_{c,l})^2 \\ \gamma_{c,l} &= \frac{\hat{x}_{c,l}}{\sqrt{\sigma_{c,l}^2 + \epsilon}} \\ \beta_{c,l} &= \gamma_{c,l} \end{aligned}$$

-- 10 --

where $a_{x,y,i}$ is the activation of the neuron at location (x, y) and channel i . I.e., each pixel in a channel is suppressed by the activations of the same pixel in its adjacent channels.

-- 11 --

Special cases The original paper recommended to only use BatchNorms after a linear transform, not after a nonlinear activation. That is, $\phi(\text{BN}(Wx+b))$, not $\text{BN}(\phi(Wx+b))$. Also, the bias b does not matter, since it would be canceled by the subsequent mean subtraction, so it is of the form $\text{BN}(Wx)$. That is, if a BatchNorm is preceded by a linear transform, then that linear transform's bias term is set to zero. For convolutional neural networks (CNNs), BatchNorm must preserve the translation-invariance of these models, meaning that it must treat all outputs of the same kernel as if they are different data points within a batch. This is sometimes called Spatial BatchNorm, or BatchNorm2D, or per-channel BatchNorm. Concretely, suppose we have a 2-dimensional convolutional layer defined by:

-- 12 --

Adaptive instance normalization Adaptive instance normalization (AdaIN) is a variant of instance normalization, designed specifically for neural style transfer with CNNs, rather than just CNNs in general. In the AdaIN method of style transfer, we take a CNN and two input images, one for content and one for style. Each image is processed through the same CNN, and at a certain layer l , AdaIN is applied. Let $x^{(l), \text{content}}$ be the activation in the content image, and $x^{(l), \text{style}}$ be the activation in the style image. Then, AdaIN first computes the mean and variance of the activations of the content image $x'^{(l)}$, then uses those as the γ, β for InstanceNorm on $x^{(l), \text{content}}$. Note that $x^{(l), \text{style}}$ itself remains unchanged. Explicitly, we have: